Containerization & Kubernetes Deployment Challenge

Name: Abhay Maurya

PRN: 202301040187

Course: CND (MDM)

GitHub Repository: https://github.com/Abhxym/kubepulse

Introduction

Containerization is an important part of cloud-native application development. A container packages an application together with its runtime and dependencies, so the same application runs the same way on any machine that has a container runtime. Kubernetes is used to run and manage these containers. It handles deployment, scaling, load balancing between replicas and restarting containers when they fail.

In this assignment I built a small web application called KubePulse, containerized it with Docker and deployed it on a local Kubernetes cluster created with Minikube. KubePulse is a Node.js application that shows details of the pod that served the request (pod name, IP address, node and namespace). Because of this, it is easy to see in the browser that the requests are being shared between the replicas and that Kubernetes restarts a pod when it crashes.

Docker is used to build an image named kubepulse:1.0. Kubernetes runs the application using a Deployment with three replicas and exposes it using a NodePort Service. The deployment is checked using kubectl commands and a web browser.

Objectives

1. Develop a simple web application and containerize it using Docker.
2. Write an optimized Dockerfile and build a Docker image named kubepulse:1.0.
3. Run and test the application in a standalone Docker container.
4. Start a local Kubernetes cluster using Minikube.
5. Deploy the application using a Kubernetes Deployment with three replicas.
6. Configure resource requests and limits, and startup, liveness and readiness probes in the Deployment.
7. Expose the application using a Kubernetes NodePort Service.
8. Verify the Deployment, Pods, Service and application access using kubectl and the browser.

9. Test load balancing, self-healing, scaling and rolling updates.
10. Maintain the source code and configuration files in GitHub.

Application Overview

KubePulse is a small Node.js web application. The server uses only the built-in modules of Node.js, so no npm packages need to be installed. The application has the following parts:

- **Server (app/server.js):** Serves the web page and provides API endpoints. It returns the pod details, the health check endpoints used by Kubernetes (/healthz and /readyz), a /metrics endpoint and a /api/chaos endpoint that crashes the pod for the self-healing test.
- **Dashboard (public/index.html):** Shows which pod served the request, container statistics (uptime, number of requests, memory, response time) and the pod details passed in by Kubernetes.
- **Load Balancing Test:** Sends 20, 60 or 150 requests to the Service and shows how many requests each pod answered.
- **Self-Healing Test:** Crashes one pod and shows an availability bar and an event log while Kubernetes restarts it.
- **Stylesheet and JavaScript (style.css, app.js):** Layout, light/dark theme and the logic for the dashboard and the tests.
- **Tests (test/server.test.js):** Seven automated tests that are run while the Docker image is being built.

Technology Stack

Technology	Purpose
Node.js 20	Runs the web server and API
HTML5, CSS3	Structure and styling of the dashboard
JavaScript	Dashboard logic, load balancing and self-healing tests
Docker	Builds and runs the container image
Kubernetes	Manages the deployment, replicas and service
Minikube	Runs a local Kubernetes cluster
kubectl	Manages and inspects Kubernetes resources
metrics-server	Provides CPU and memory usage for kubectl top and the HPA
PowerShell	Deployment automation through deploy.ps1
Git and GitHub	Version control and source code hosting

Project Structure

The project directory contains the following files:

```
kubepulse/
├── Dockerfile
├── .dockerignore
├── deploy.ps1
├── deploy.sh
├── package.json
├── README.md
├── app/
│   └── server.js
├── public/
│   ├── index.html
│   ├── style.css
│   ├── app.js
│   └── favicon.svg
├── test/
│   └── server.test.js
├── k8s/
│   ├── 00-namespace.yaml
│   ├── 01-configmap.yaml
│   ├── 02-deployment.yaml
│   ├── 03-service.yaml
│   └── 04-hpa.yaml
└── docs/
    └── evidence/
```

File descriptions:

- **Dockerfile:** Defines how the Docker image is built (multi-stage build).
- **.dockerignore:** Excludes files that are not needed in the image, such as the k8s folder and documentation.
- **deploy.ps1 / deploy.sh:** Scripts that build the image, load it into Minikube and apply the manifests.
- **app/server.js:** The Node.js web server.
- **public/:** The HTML, CSS and JavaScript files of the dashboard.
- **test/server.test.js:** Automated tests for the server.
- **00-namespace.yaml:** Creates the kubepulse namespace.
- **01-configmap.yaml:** Stores the application settings.
- **02-deployment.yaml:** Defines the Kubernetes Deployment.
- **03-service.yaml:** Defines the Kubernetes Service.
- **04-hpa.yaml:** Defines the Horizontal Pod Autoscaler.
- **docs/evidence/:** Screenshots and command outputs.

The manifest files are numbered so that kubectl apply -f k8s/ creates the namespace and the ConfigMap before the Deployment and Service.

Dockerfile and Image Creation

The Dockerfile uses a multi-stage build with node:20-alpine as the base image. Alpine based images are small, which keeps the final image size low. The base image is pinned using its digest so that every build uses exactly the same image.

The first stage (test) copies the source code and runs the automated tests. If any test fails, the build stops and no image is created. The second stage (runtime) starts again from a clean base image and copies only the files that are needed to run the application, so the test files are not included in the final image. The container runs as the non-root user node, and a HEALTHCHECK is added so that Docker can report whether the container is healthy.

Dockerfile

```
# KubePulse Dockerfile (multi-stage build)

# Base image is pinned by digest so every build uses exactly the same Node.js
image.

# Stage 1: run the tests. If a test fails the build stops here.
FROM node:20-
alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c58a04632e6c0fb8363f609372293
AS test
WORKDIR /src
```

```
COPY package.json ./
COPY app ./app
COPY public ./public
COPY test ./test
RUN node --test test/

# Stage 2: runtime image with only the files needed to run the app
FROM node:20-
alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c58a04632e6c0fb8363f609372293
AS runtime

LABEL maintainer="Abhay Maurya" \
      description="KubePulse - containerized Node.js app for the Kubernetes
assignment" \
      version="1.0"

ENV NODE_ENV=production \
    PORT=8080

WORKDIR /app

COPY --from=test /src/package.json ./
COPY --from=test /src/app ./app
COPY --from=test /src/public ./public

# run as the non-root "node" user that comes with the base image
USER node

EXPOSE 8080

HEALTHCHECK --interval=15s --timeout=3s --start-period=5s --retries=3 \
  CMD wget -qO- http://127.0.0.1:8080/healthz || exit 1

CMD ["node", "app/server.js"]
```

Optimizations used in the Dockerfile:

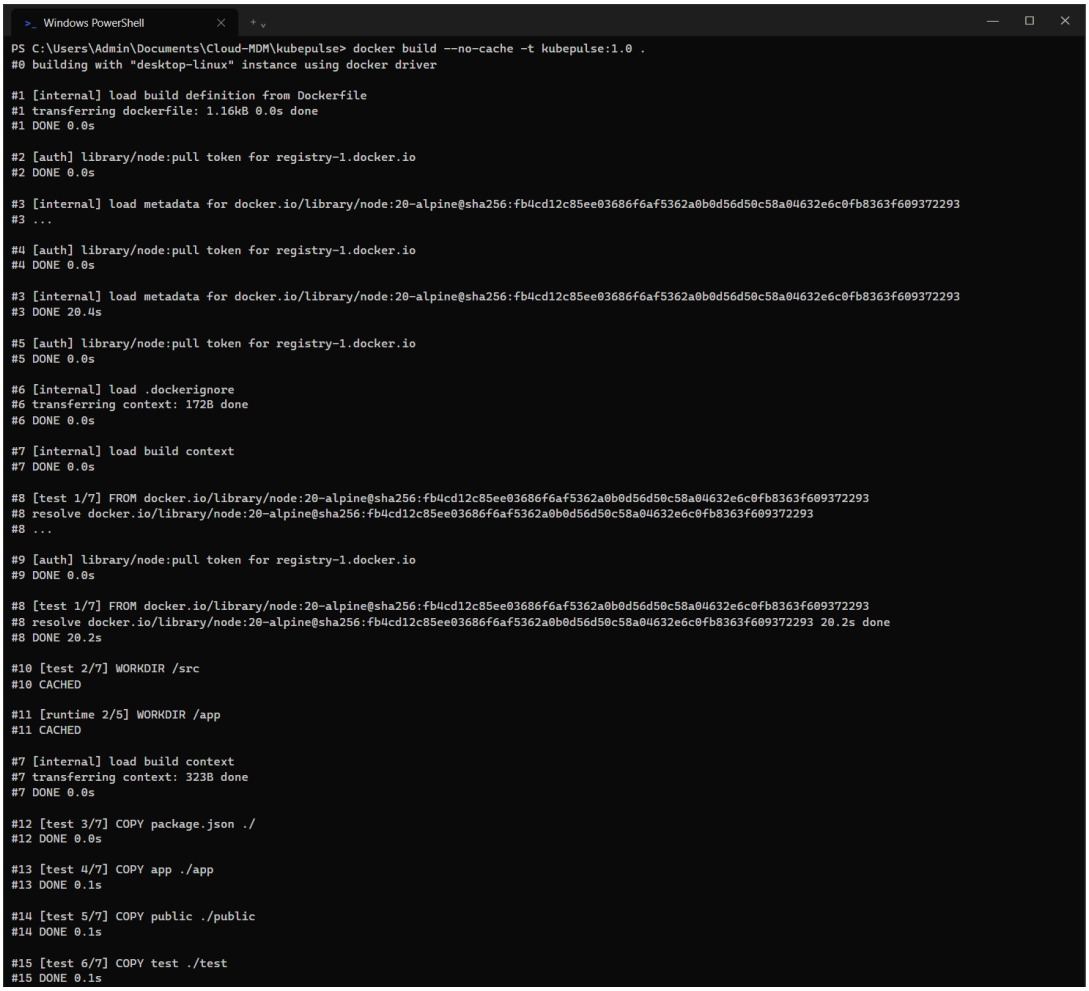
- Multi-stage build, so tests run during the build and are not shipped in the final image.
- Small Alpine base image and no npm dependencies.
- .dockerignore to keep the build context small.
- Application runs as a non-root user.
- HEALTHCHECK instruction for container health.
- CMD in exec form, so Node.js receives the stop signal (SIGTERM) and can shut down gracefully.

Building the Docker Image

Open PowerShell in the project root directory and execute:

docker build -t kubepulse:1.0 .

The output shows both stages of the build. In step #16 the seven tests are executed and all of them pass.



```
#16 [test 7/7] RUN node --test test/
#16 0.399 TAP version 13
#16 0.570 # Subtest: liveness probe returns 200
#16 0.571 ok 1 - liveness probe returns 200
#16 0.572   ---
#16 0.572   duration_ms: 89.05579
#16 0.572   ...
#16 0.582 # Subtest: readiness probe is 503 until warm, then 200
#16 0.582 ok 2 - readiness probe is 503 until warm, then 200
#16 0.582   ---
#16 0.582   duration_ms: 12.707769
#16 0.582   ...
#16 0.597 # Subtest: /api/info reports pod metadata
#16 0.598 ok 3 - /api/info reports pod metadata
#16 0.598   ---
#16 0.598   duration_ms: 12.937876
#16 0.598   ...
#16 0.609 # Subtest: /api/hit identifies the serving pod
#16 0.609 ok 4 - /api/hit identifies the serving pod
#16 0.609   ---
#16 0.609   duration_ms: 14.273
#16 0.609   ...
#16 0.618 # Subtest: dashboard is served
#16 0.618 ok 5 - dashboard is served
#16 0.618   ---
#16 0.618   duration_ms: 9.01272
#16 0.618   ...
#16 0.625 # Subtest: path traversal is blocked
#16 0.625 ok 6 - path traversal is blocked
#16 0.625   ---
#16 0.625   duration_ms: 6.504775
#16 0.625   ...
#16 0.632 # Subtest: metrics are exposed in Prometheus format
#16 0.632 ok 7 - metrics are exposed in Prometheus format
#16 0.632   ---
#16 0.632   duration_ms: 7.690552
#16 0.632   ...
#16 0.702 1..7
#16 0.702 # tests 7
#16 0.702 # suites 0
#16 0.702 # pass 7
#16 0.702 # fail 0
#16 0.702 # cancelled 0
#16 0.702 # skipped 0
#16 0.702 # todo 0
#16 0.703 # duration_ms 332.234674
#16 DONE 0.7s

#17 [runtime 3/5] COPY --from=test /src/package.json ./
#17 DONE 0.1s

#18 [runtime 4/5] COPY --from=test /src/app ./app
#18 DONE 0.1s

#19 [runtime 5/5] COPY --from=test /src/public ./public
#19 DONE 0.1s

#20 exporting to image
#20 exporting layers
#20 exporting layers 0.2s done
#20 exporting manifest sha256:bcd0a92bee1bab76a4f7a169baea44461499dc84ebea40c212d55676cdaacd8f 0.0s done
#20 exporting config sha256:ad40732d2349ed60f5fc11b846997b17664723584d92d2a03567e5175fa9e2f0 0.0s done
#20 exporting attestation manifest sha256:8cafffb9306174fdd82f0419be437be92f1cfc6a5c52441a58104f347de0239f 0.0s done
#20 exporting manifest list sha256:1c05b6d612f16beda8d45550ba9d03d7154fc13be94304110cde0d1a3c00542d 0.0s done
#20 naming to docker.io/library/kubepulse:1.0 done
#20 unpacking to docker.io/library/kubepulse:1.0
#20 unpacking to docker.io/library/kubepulse:1.0 0.1s done
#20 DONE 0.4s

PS C:\Users\Admin\Documents\Cloud-MDM\kubepulse>
```

Verify the image:

docker images kubepulse

```
Windows PowerShell
PS C:\Users\Admin\Documents\Cloud-MDM\kubepulse> docker images kubepulse
REPOSITORY   TAG       IMAGE ID       CREATED              SIZE
kubepulse    1.0       1c05b6d612f1   About a minute ago   193MB

PS C:\Users\Admin\Documents\Cloud-MDM\kubepulse>
```

The image appears with the repository name kubepulse and tag 1.0.

Container Execution and Local Testing

Before deploying the application to Kubernetes, the Docker image is tested as a standalone container.

Running the Container

The following command maps port 8080 on the host to port 8080 inside the container:

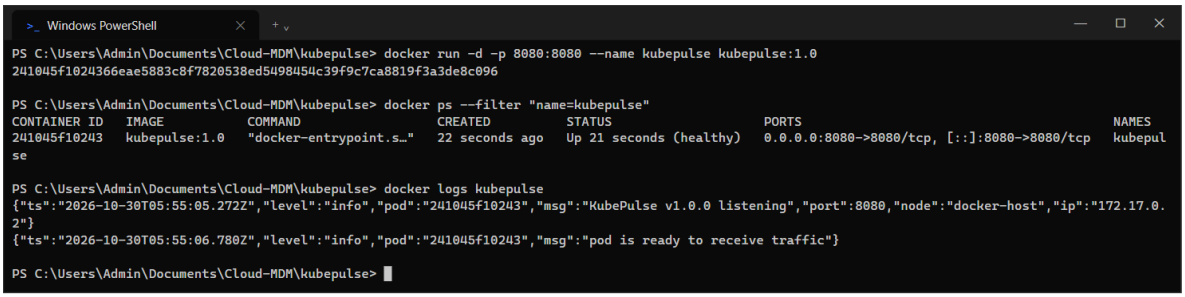
docker run -d -p 8080:8080 --name kubepulse kubepulse:1.0

Verify that the container is running:

docker ps --filter "name=kubepulse"

Inspect the container logs:

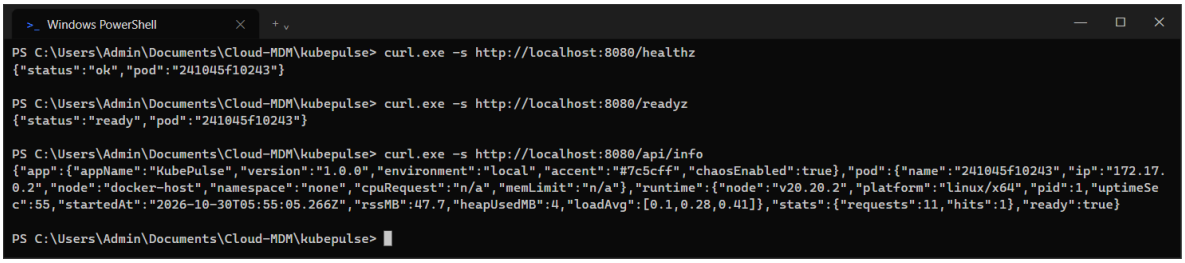
docker logs kubepulse



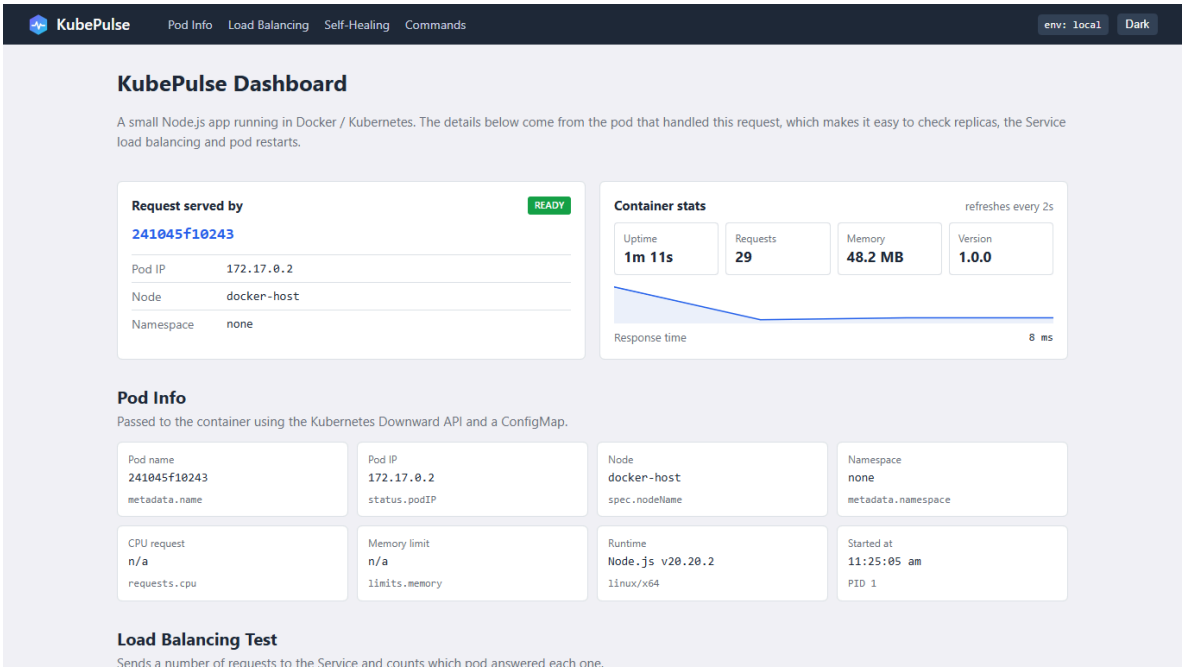
The STATUS column shows (healthy), which means the HEALTHCHECK is passing. The logs are written in JSON format.

Testing the Application

The health check and API endpoints were tested using curl:



The application was then opened in the browser at http://localhost:8080/. When the application runs only in Docker, there is no Kubernetes information, so the pod name shows the container ID, the node shows docker-host and the namespace shows none.



Kubernetes Cluster Setup

Minikube is used to run a local Kubernetes cluster. Docker Desktop provides the container runtime for the cluster.

Starting Minikube

Ensure Docker Desktop is running and execute:

minikube start --driver=docker

The metrics-server addon is enabled so that kubectl top and the HPA can read CPU usage:

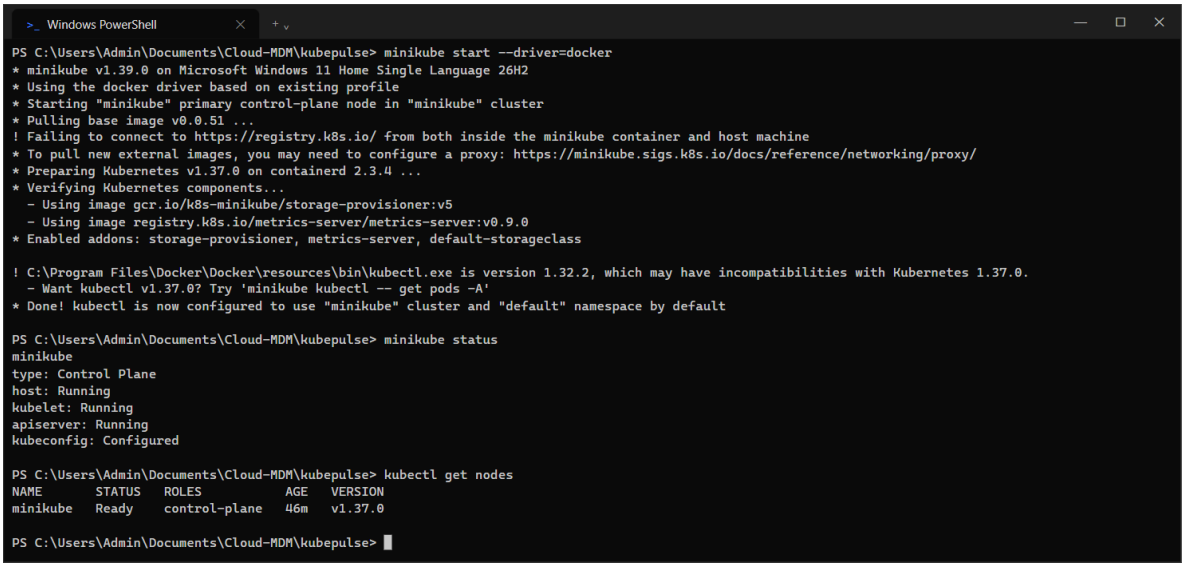
minikube addons enable metrics-server

Check the cluster status:

minikube status

Verify that Kubernetes recognizes the node:

kubectl get nodes



The node shows a Ready status, so the application can be deployed.

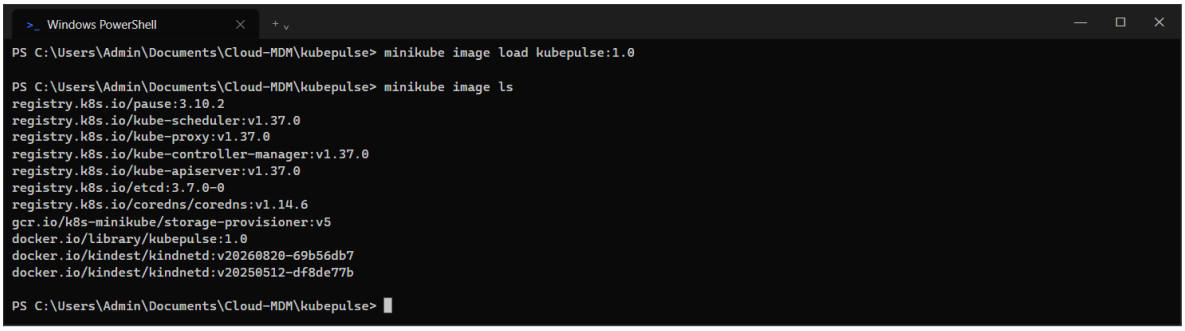
Loading the Docker Image into Minikube

Minikube has its own container runtime, so the image built on the host has to be loaded into it:

minikube image load kubepulse:1.0

Verify that the image is available:

minikube image ls



The output contains docker.io/library/kubepulse:1.0. The Deployment uses imagePullPolicy: IfNotPresent, so Kubernetes uses this local image and does not try to download it from a registry.

Kubernetes Deployment Configuration

A Kubernetes Deployment manages the application Pods and maintains the desired replica count.

The Deployment is configured with three replicas and a rolling update strategy (maxSurge: 1, maxUnavailable: 0), so during an update a new pod is started before an old pod is removed. It also sets resource requests and limits, startup, liveness and readiness probes, a security context, and environment variables from the ConfigMap and the Downward API.

k8s/02-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: kubepulse
  namespace: kubepulse
  labels:
    app: kubepulse
    tier: web
    app.kubernetes.io/name: kubepulse
    app.kubernetes.io/version: "1.0.0"
spec:
  # Three replicas = high availability; the Service load-balances across
them.
  replicas: 3
  revisionHistoryLimit: 5
  selector:
    matchLabels:
      app: kubepulse
  # Zero-downtime rolling updates: add one new pod before removing an old
one.
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: kubepulse
        tier: web
        version: "1.0.0"
    spec:
      terminationGracePeriodSeconds: 20
      # Prefer spreading replicas over different nodes when more than one
exists.
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: kubernetes.io/hostname
          whenUnsatisfiable: ScheduleAnyway
          labelSelector:
            matchLabels:
              app: kubepulse
      securityContext:
        runAsNonRoot: true
        runAsUser: 1000
        runAsGroup: 1000
```

```
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: kubepulse
          image: kubepulse:1.0
          # Image is loaded into minikube locally, never pulled from a
registry.
          imagePullPolicy: IfNotPresent
          ports:
            - name: http
              containerPort: 8080
              protocol: TCP

          # Non-secret settings come from the ConfigMap...
          envFrom:
            - configMapRef:
                name: kubepulse-config
          # ...and pod identity comes from the Downward API.
          env:
            - name: POD_NAME
              valueFrom:
                fieldRef:
                  fieldPath: metadata.name
            - name: POD_NAMESPACE
              valueFrom:
                fieldRef:
                  fieldPath: metadata.namespace
            - name: POD_IP
              valueFrom:
                fieldRef:
                  fieldPath: status.podIP
            - name: NODE_NAME
              valueFrom:
                fieldRef:
                  fieldPath: spec.nodeName
            - name: CPU_REQUEST
              valueFrom:
                resourceFieldRef:
                  containerName: kubepulse
                  resource: requests.cpu
                  divisor: 1m
            - name: MEM_LIMIT
              valueFrom:
                resourceFieldRef:
                  containerName: kubepulse
                  resource: limits.memory
                  divisor: 1Mi

          # Guaranteed minimum (requests) and hard ceiling (limits) per
container.
          resources:
            requests:
              cpu: "50m"
              memory: "64Mi"
```

```
            limits:
              cpu: "250m"
              memory: "128Mi"

          # Startup probe protects slow starts from being killed by liveness.
          startupProbe:
            httpGet:
              path: /healthz
              port: http
            periodSeconds: 2
            failureThreshold: 15
          # Liveness: restart the container if the process hangs.
          livenessProbe:
            httpGet:
              path: /healthz
              port: http
            periodSeconds: 10
            timeoutSeconds: 2
            failureThreshold: 3
          # Readiness: only route Service traffic to warmed-up, non-draining
pods.
          readinessProbe:
            httpGet:
              path: /readyz
              port: http
            initialDelaySeconds: 2
            periodSeconds: 3
            timeoutSeconds: 2
            failureThreshold: 2

          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities:
              drop: ["ALL"]
```

Main settings of the Deployment:

Setting	Value	Purpose
replicas	3	Three copies of the application are always running
strategy	RollingUpdate (maxSurge 1, maxUnavailable 0)	Updates without downtime
resources.requests	cpu 50m, memory 64Mi	Minimum resources reserved for each pod
resources.limits	cpu 250m, memory 128Mi	Maximum resources each pod can use
startupProbe	GET /healthz	Gives the container time to start
livenessProbe	GET /healthz every 10s	Restarts the container if it stops responding
readinessProbe	GET /readyz every 3s	Sends traffic only to pods that are ready
envFrom	kubepulse-config	Loads settings from the ConfigMap
Downward API	POD_NAME, POD_IP, NODE_NAME, etc.	Passes pod details to the application
securityContext	runAsNonRoot, readOnlyRootFilesystem	Runs the container with fewer privileges

k8s/00-namespace.yaml

```
# Isolates every KubePulse resource in its own namespace.
apiVersion: v1
kind: Namespace
metadata:
  name: kubepulse
  labels:
    app.kubernetes.io/part-of: kubepulse
```

k8s/01-configmap.yaml

```
# Externalised configuration: change behaviour without rebuilding the image.
apiVersion: v1
kind: ConfigMap
metadata:
  name: kubepulse-config
  namespace: kubepulse
  labels:
    app: kubepulse
data:
  APP_NAME: "KubePulse"
```

```
  APP_VERSION: "1.0.0"
  APP_ENV: "minikube"
  ENABLE_CHAOS: "true"
```

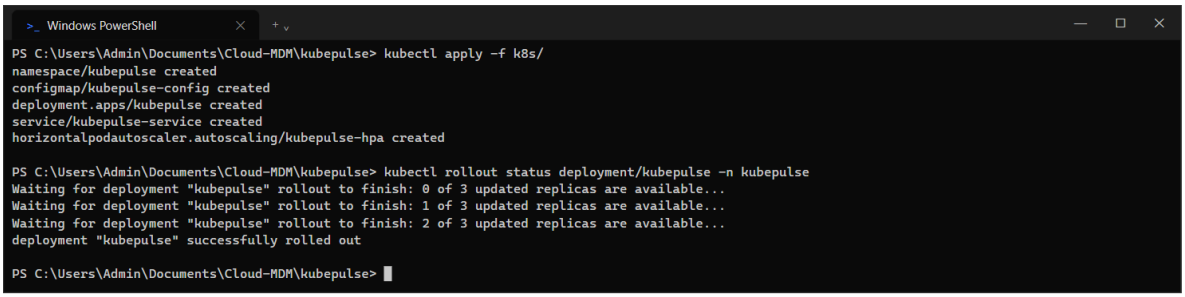
Applying the Deployment

Execute:

kubectl apply -f k8s/

Wait for the rollout to complete:

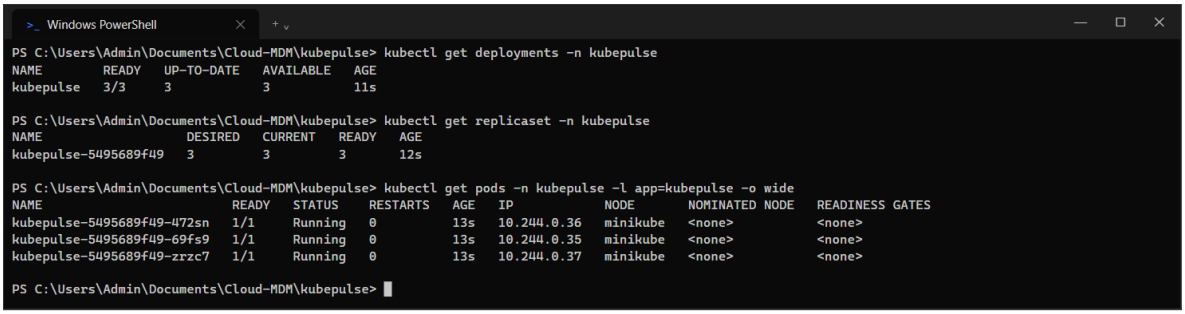
kubectl rollout status deployment/kubepulse -n kubepulse



Check the Deployment, ReplicaSet and Pods:

kubectl get deployments -n kubepulse

kubectl get pods -n kubepulse -l app=kubepulse -o wide



Three Pods are in the Running state and each Pod shows 1/1 ready. Each Pod has its own IP address.

Kubernetes Service Configuration

A Kubernetes Service provides a stable access point for the application Pods. Pods can be deleted and recreated with new IP addresses, but the Service address stays the same. The project uses a NodePort Service which exposes the application on port 30080 of the Minikube node and forwards traffic to port 8080 of the Pods.

k8s/03-service.yaml

```
# Stable network endpoint in front of all ready KubePulse pods.
apiVersion: v1
kind: Service
metadata:
  name: kubepulse-service
  namespace: kubepulse
  labels:
    app: kubepulse
spec:
  type: NodePort
  # Must match the pod template labels in the Deployment.
  selector:
    app: kubepulse
  ports:
    - name: http
      protocol: TCP
      port: 80          # Service (ClusterIP) port
      targetPort: http  # named containerPort 8080 on each pod
      nodePort: 30080   # exposed on every node: <NodeIP>:30080
```

Applying and Verifying the Service

The Service is created together with the other manifests by kubectl apply -f k8s/. Verify the Service:

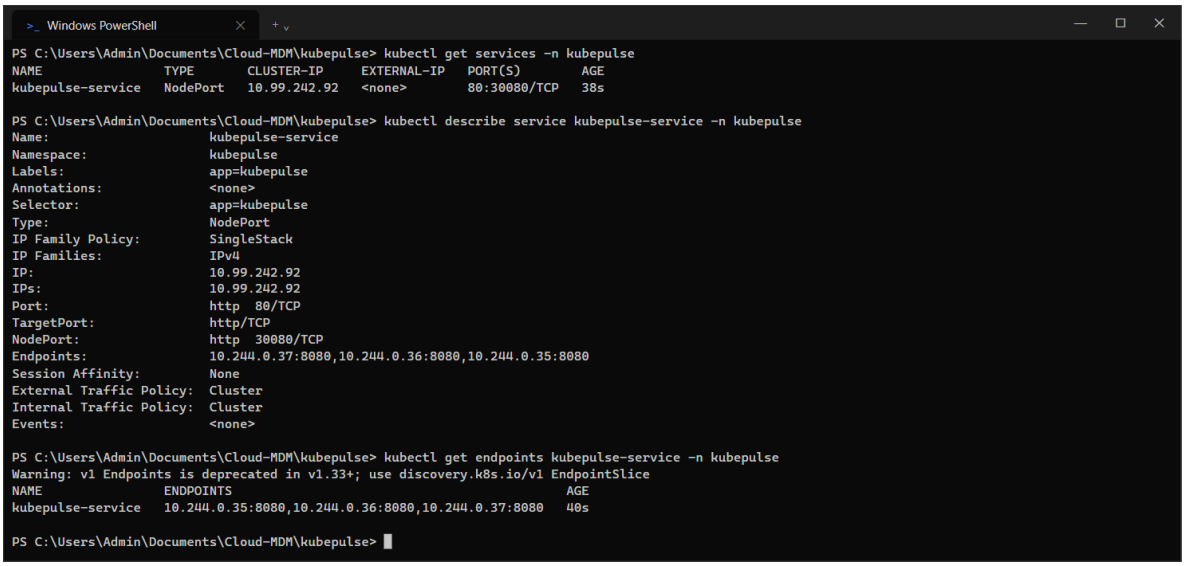
kubectl get services -n kubepulse

Inspect its configuration:

kubectl describe service kubepulse-service -n kubepulse

Check its endpoints:

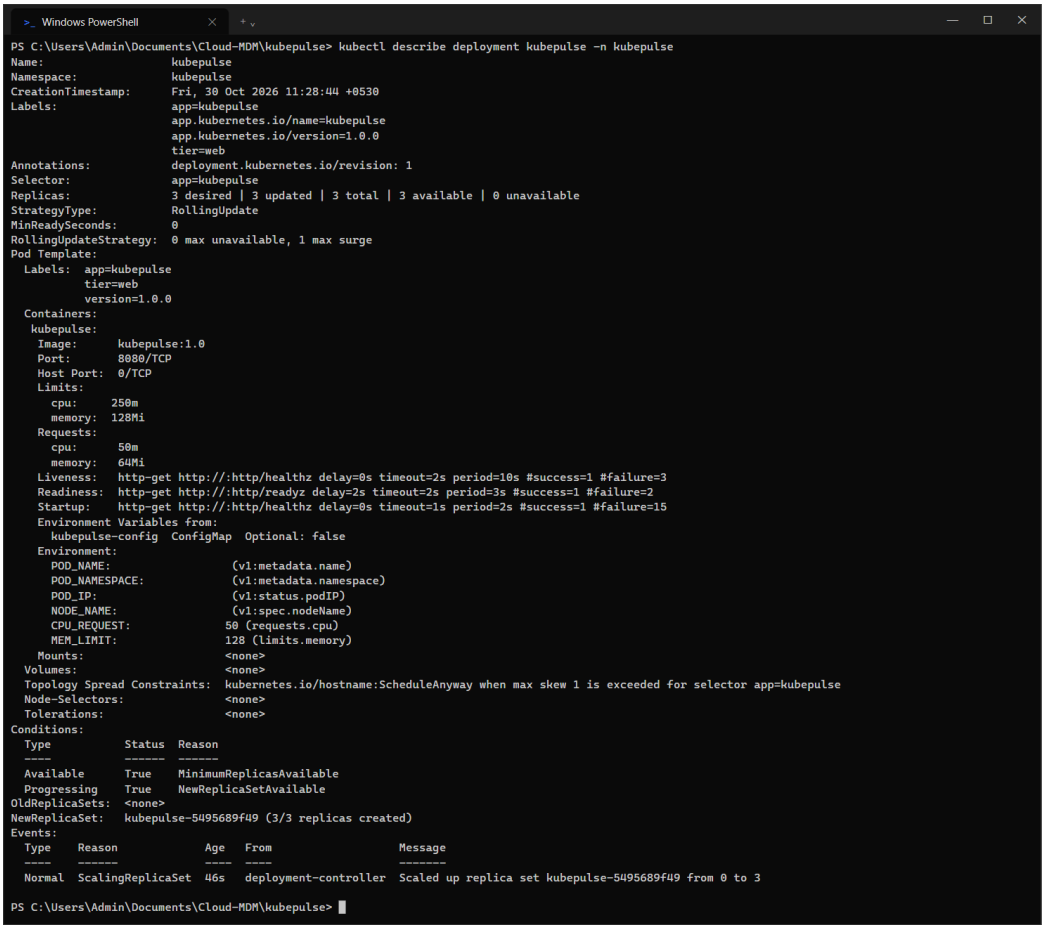
kubectl get endpoints kubepulse-service -n kubepulse



The Service selector app=kubepulse matches the Pod labels from the Deployment. The endpoints list contains the IP addresses of the three Pods, which means all three Pods are ready and receive traffic.

The full details of the Deployment can be checked with kubectl describe:

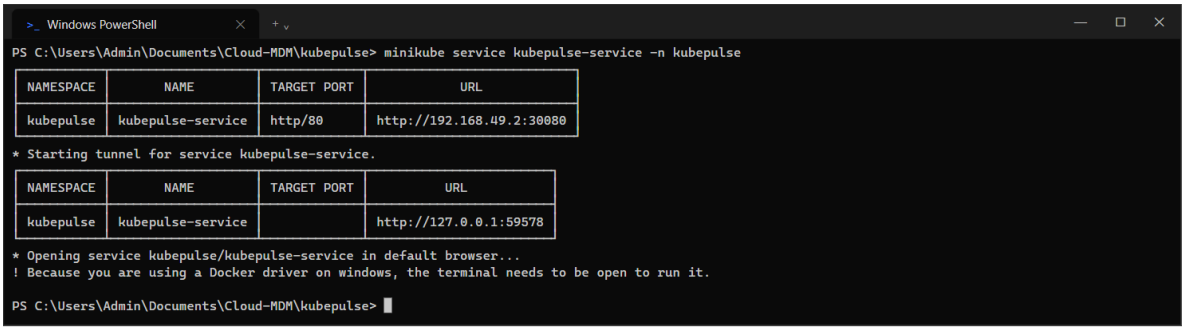
kubectl describe deployment kubepulse -n kubepulse



Accessing the Application Through Kubernetes

The application can be accessed through the Minikube service command:

minikube service kubepulse-service -n kubepulse



Because the Docker driver is used on Windows, Minikube starts a tunnel and gives a local URL (here http://127.0.0.1:59578). The terminal has to stay open while the application is being used. The port changes every time the command is run again.

Load Balancing

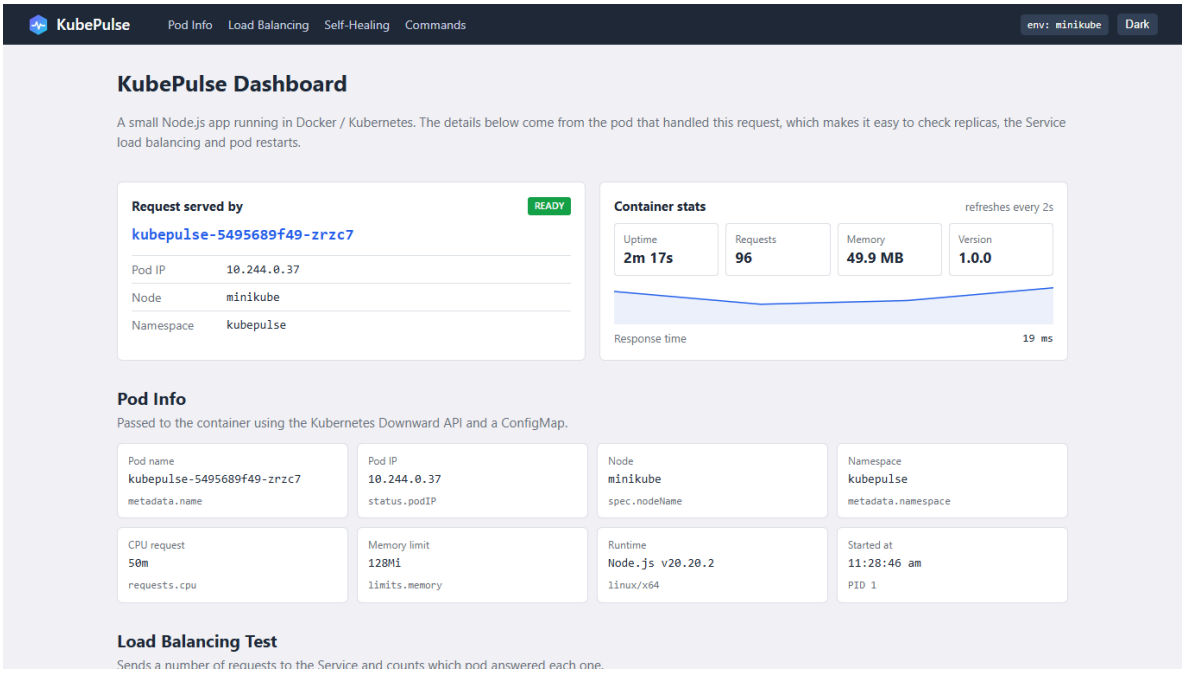
To check that the Service shares the requests between the Pods, 30 requests were sent to the /api/hit endpoint and the responses were grouped by pod name:



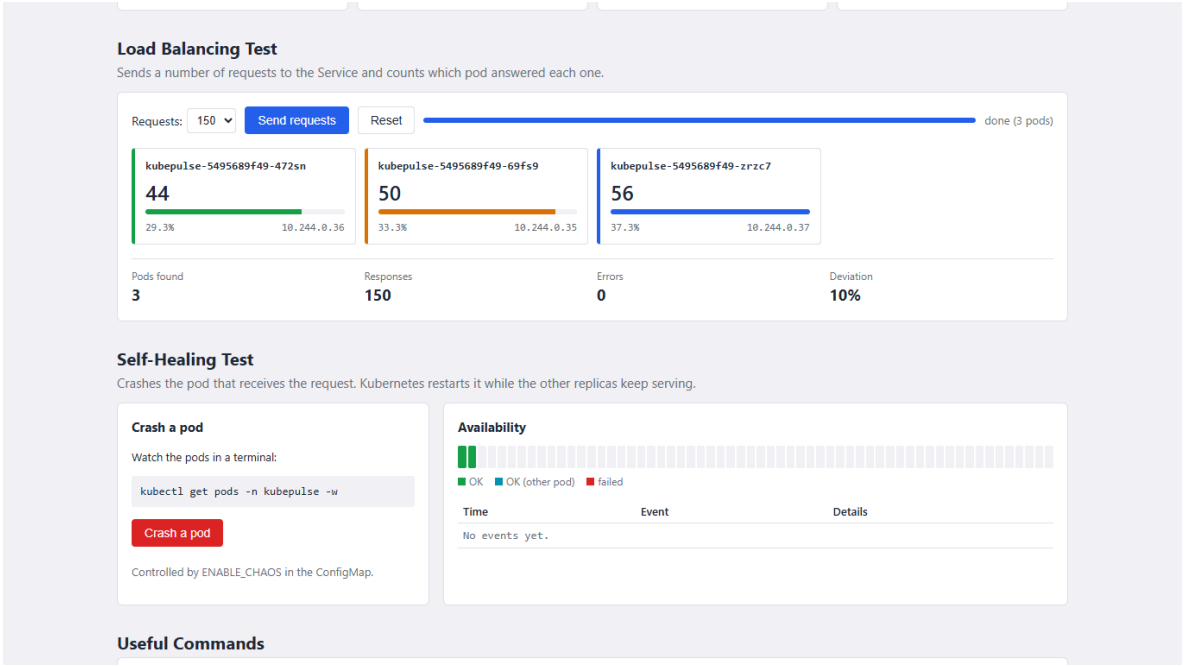
All three Pods answered requests (13, 9 and 8), so the Service is load balancing between the replicas.

Application Running Through Kubernetes

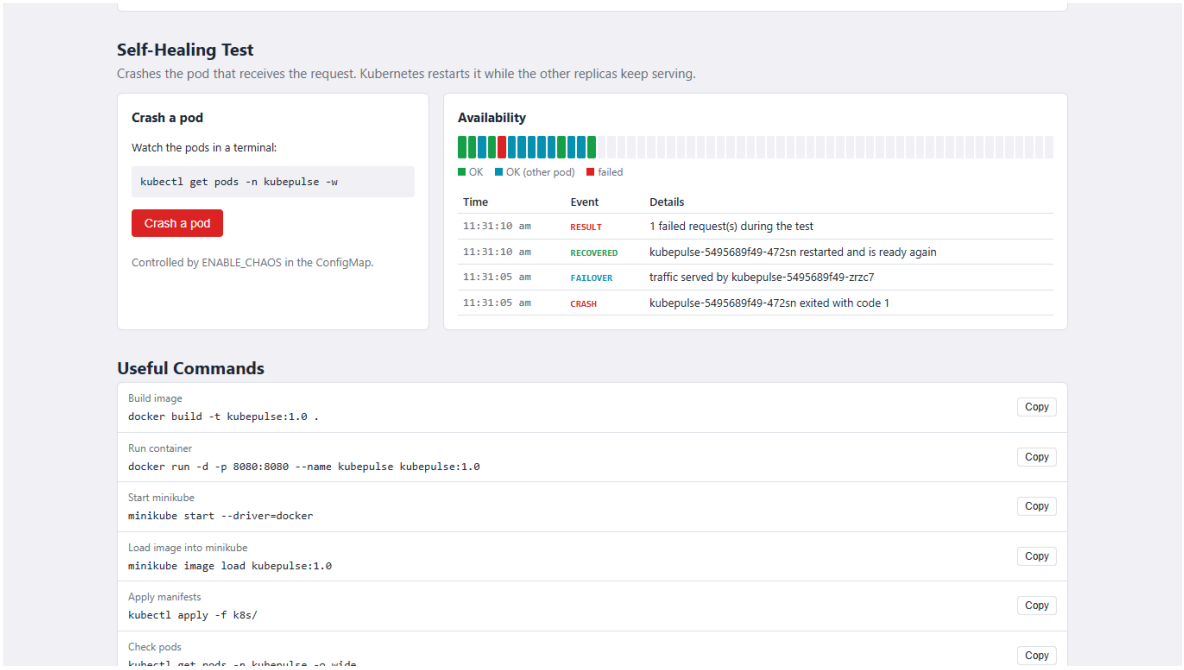
The dashboard opened through the Service URL. The page shows env: minikube (from the ConfigMap), the pod name, the pod IP, node minikube and namespace kubepulse. The CPU request (50m) and memory limit (128Mi) come from the Deployment through the Downward API.



Load Balancing Test in the dashboard with 150 requests. The requests were divided between the three pods:



Self-Healing Test in the dashboard. After clicking Crash a pod, the requests were served by another pod (FAILOVER) and the crashed pod was restarted by Kubernetes (RECOVERED). Only one request failed during the test.

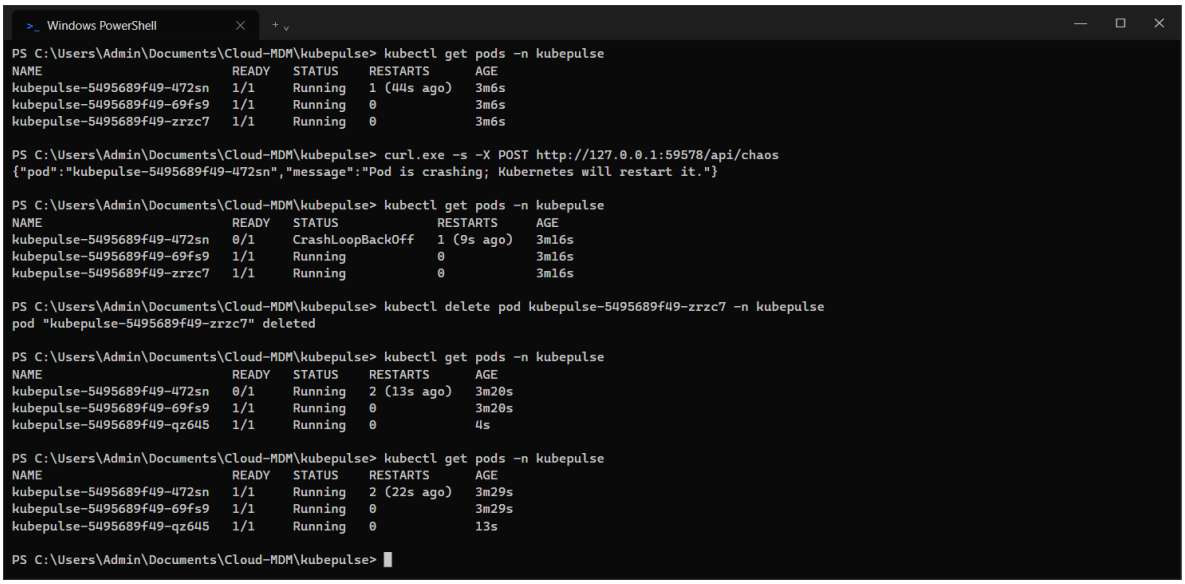


Self-Healing

Self-healing was also tested from the terminal. First a pod was crashed using the /api/chaos endpoint. Kubernetes restarted the container and the RESTARTS count increased. Because this pod had already been crashed once from the dashboard, it was shown in CrashLoopBackOff for a few seconds (Kubernetes waits a little longer before restarting a container that crashes repeatedly) and then it was Running again.

Then a pod was deleted. The ReplicaSet created a new pod immediately to keep the number of replicas at three.

kubectl delete pod kubepulse-5495689f49-zrzc7 -n kubepulse



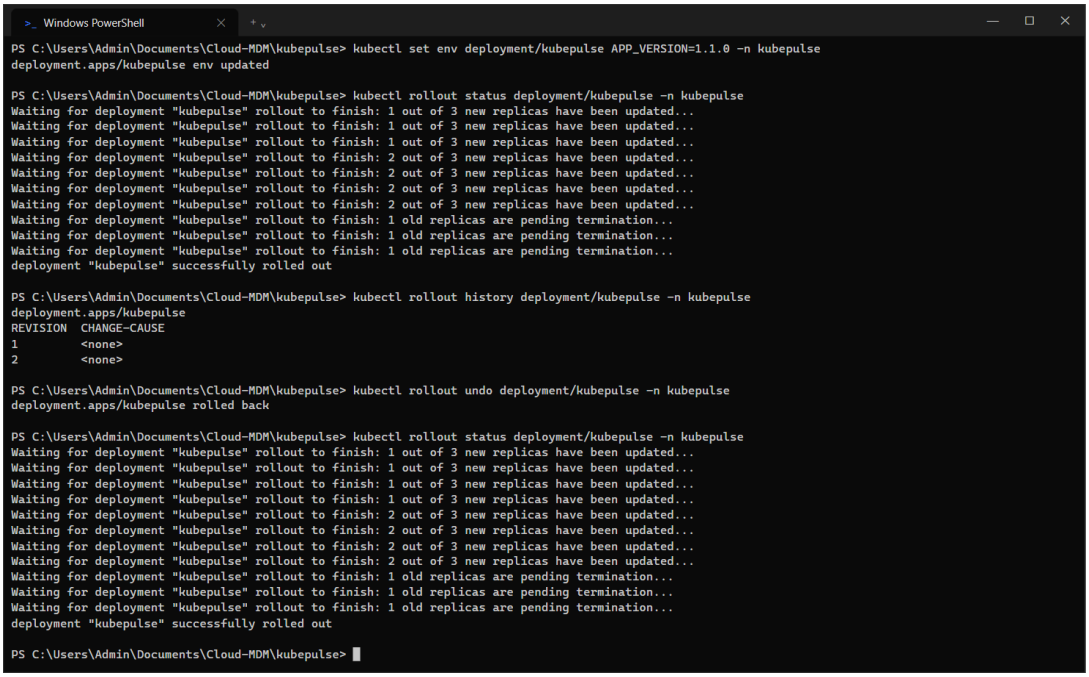
Rolling Update and Rollback

A change was rolled out by setting a new value for APP_VERSION. Kubernetes replaced the pods one by one, so the application stayed available during the update. The update was then rolled back to the previous version with kubectl rollout undo.

kubectl set env deployment/kubepulse APP_VERSION=1.1.0 -n kubepulse

kubectl rollout history deployment/kubepulse -n kubepulse

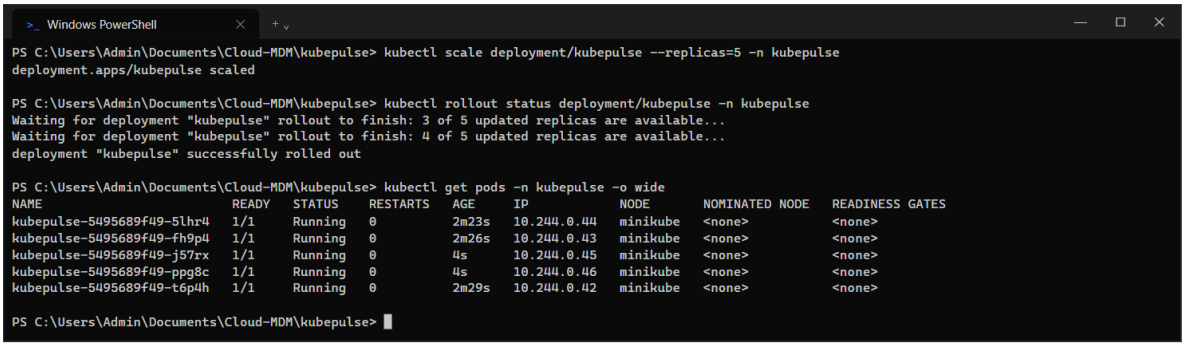
kubectl rollout undo deployment/kubepulse -n kubepulse



Scaling

The Deployment can be scaled by changing the number of replicas:

kubectl scale deployment/kubepulse --replicas=5 -n kubepulse



Two new pods were created and all five pods are Running. A Horizontal Pod Autoscaler is also configured, so the number of replicas can change automatically between 3 and 6 based on CPU usage.

k8s/04-hpa.yaml

```
# Scales between 3 and 6 replicas based on CPU usage.
# Requires metrics-server:  minikube addons enable metrics-server
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: kubepulse-hpa
  namespace: kubepulse
  labels:
    app: kubepulse
```

```
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: kubepulse
  minReplicas: 3
  maxReplicas: 6
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

Verification and Monitoring

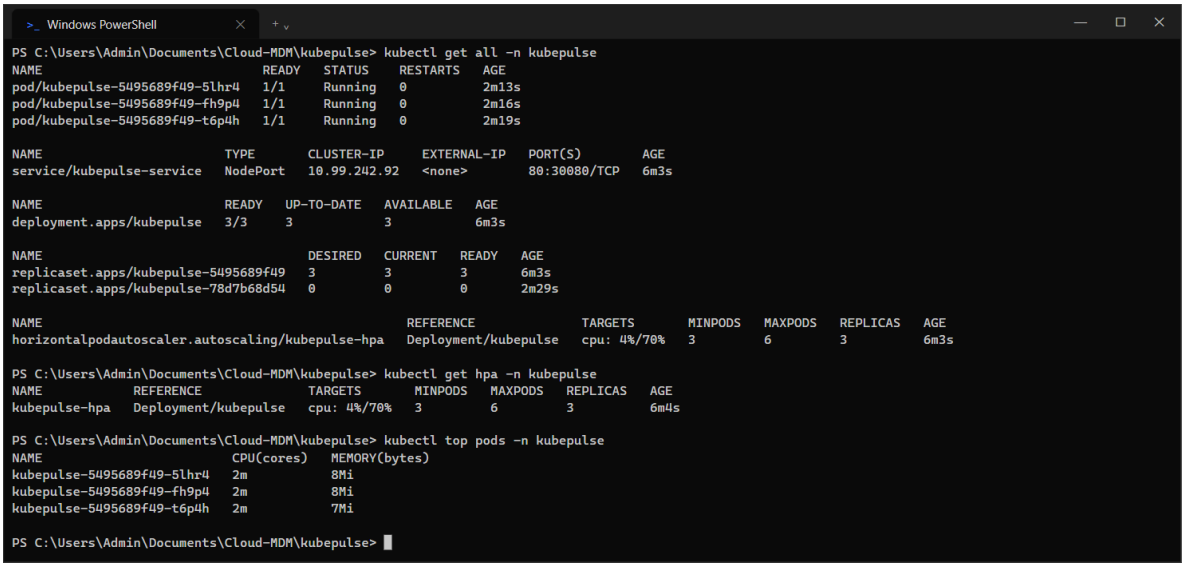
Kubernetes provides commands to inspect the deployment, Pods, Service and application logs.

Viewing Kubernetes Resources

kubectl get all -n kubepulse

kubectl get hpa -n kubepulse

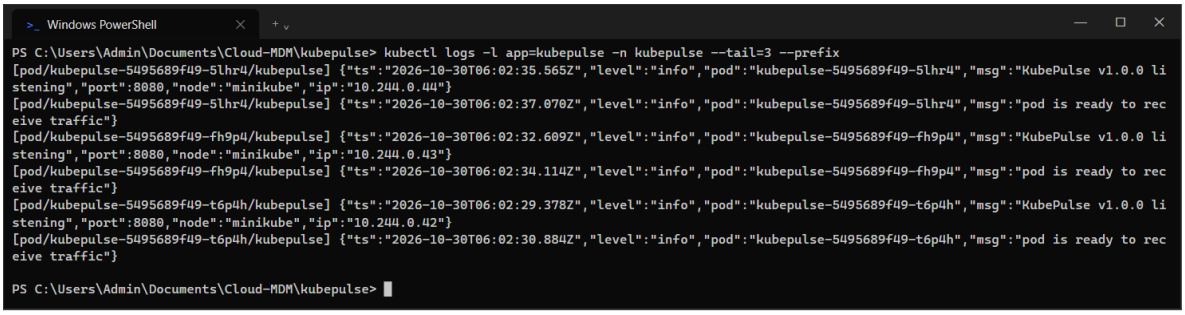
kubectl top pods -n kubepulse



The HPA shows the current CPU usage (4%) against the target of 70%, and kubectl top shows that each pod uses around 8 to 9 MiB of memory, which is well below the 128Mi limit.

Inspecting Pod Logs

kubectl logs -l app=kubepulse -n kubepulse --tail=3 --prefix



These commands help verify the current state of the deployment, inspect configuration details and troubleshoot problems.

The liveness probe checks whether the application is still responding. The readiness probe checks whether the Pod should receive traffic from the Service. Resource requests and limits specify the CPU and memory reserved for and allowed to each container.

Cloud-Native Concepts Used

- **Containerization:** the application and its runtime are packaged in one Docker image.
- **Declarative configuration:** the required state is written in YAML files and applied with kubectl.
- **Replication:** the Deployment keeps three replicas running.
- **Service discovery and load balancing:** the Service gives a fixed address and spreads requests over the pods.
- **Self-healing:** crashed containers are restarted and deleted pods are recreated.
- **Health checks:** startup, liveness and readiness probes.
- **Resource management:** CPU and memory requests and limits.
- **Scaling:** manual scaling with kubectl scale and automatic scaling with the HPA.
- **Rolling updates and rollback:** updates without downtime.
- **Configuration outside the image:** ConfigMap and Downward API.

GitHub Repository

The project source code and Kubernetes configuration are maintained in the following GitHub repository:

Repository URL:
https://github.com/Abhxym/kubepulse

The repository includes the Dockerfile, application source code, tests, Kubernetes manifests, deployment scripts, screenshots and README file. Using version control makes

it possible to keep track of changes and to set up the same project on another machine.

Conclusion

In this assignment the KubePulse web application was containerized using Docker and deployed on Kubernetes using Minikube.

The application was packaged into a Docker image named kubepulse:1.0 using a multi-stage Dockerfile, and all tests passed during the build. The image was tested as a standalone container and then loaded into Minikube and deployed using a Deployment with three replicas, resource requests and limits, and health probes.

A NodePort Service was used to expose the application. kubectl commands were used to check the node, Deployment, Pods, Service, endpoints and logs, and the application was opened in the browser through the URL provided by Minikube. The tests showed that the Service shares requests between the pods, that Kubernetes restarts crashed pods and replaces deleted pods, and that the Deployment can be scaled, updated and rolled back without stopping the application.

This assignment gave practical experience with Docker image creation, running containers, Kubernetes Deployments and Services, resource configuration, health probes and accessing an application through a local Kubernetes cluster.